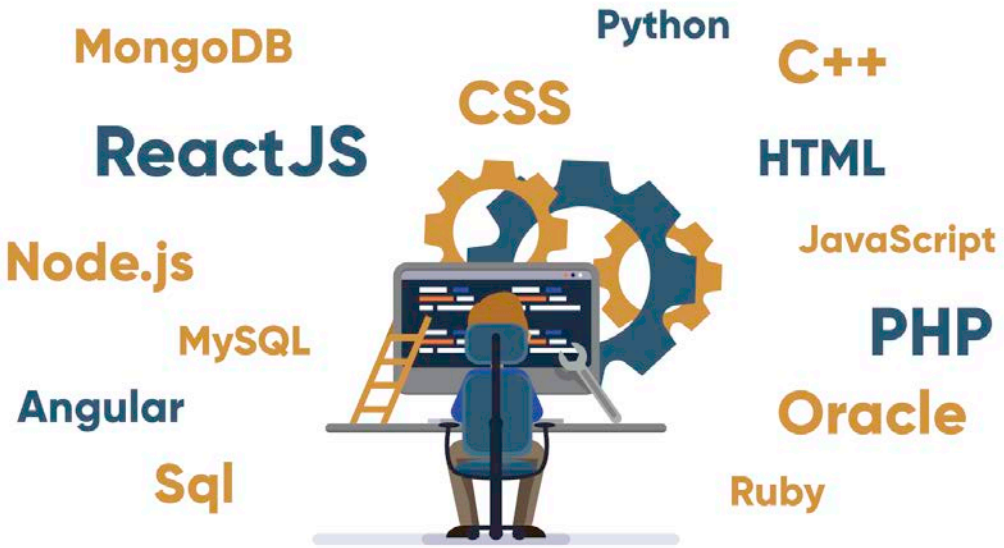
FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE 1 - INTRODUCTION

WHAT IS FULL-STACK DEVELOPMENT?

- Full-stack development refers to building both the front-end (user interface) and back-end (server-side logic) of a web application.
- Full-stack developers are comfortable working with all layers of software development. They handle everything from client-side scripting and styling to server-side programming and database management.



FRONT-END VS. BACK-END

- **Front-End (Client-Side):** The front-end is the part of the application that users directly interact with (essentially the **user interface** running in the browser).
- **Back-End (Server-Side):** The back-end is the behind-the-scenes engine of the application. It includes the server, application logic, and database that together handle data processing, business logic, and data storage.

HOW THEY INTERACT?

- Front-end and back-end communicate via HTTP requests and APIs.
- The front-end (e.g., a React or Angular app running in the browser) will request data from the back-end (e.g., an Express or Django server) through endpoints.
- The back-end returns JSON or HTML data, which the front-end then presents to the user.

FULL-STACK PERSPECTIVE

- A full-stack developer understands both sides!
- This means knowing how to create a good user experience on the front-end **and** how to implement the logic and database interactions on the back-end.
- Having this dual knowledge helps in debugging and in designing features.

COURSE STRUCTURE

- **1. Introduction (Week 1):** Overview of full-stack development. We introduce key concepts, the web development landscape, and course logistics.
- **2. Front-End Technologies (Weeks 2-5):** Covering the essentials of client-side development. We will dive into:
  - **JavaScript** – the primary language of the web browser.
  - **React** – a popular library for building user interfaces using components.
  - **Angular** – a powerful full-featured framework for front-end development.

COURSE STRUCTURE

- **3. Server-Side Programming (Weeks 6-9):** Focusing on building the back-end of applications. Topics include:
  - **Node.js** – JavaScript runtime for server-side scripting.
  - **Express** – a minimalist web framework for Node.js to create APIs and web servers.
  - **Python/Django** – using Python with the Django framework to build robust web applications.
- **4. Database Management (Weeks 10-11):** Storing and managing data for applications.
  - **SQL Databases** – relational databases (e.g. MySQL) and structured query language.
  - **NoSQL Databases** – non-relational databases (e.g. MongoDB) for flexible schemas and big data needs.

COURSE STRUCTURE

- **5. DevOps Practices (Week 12):** Introduction to development operations (version control, continuous integration/deployment, automation, and other practices to streamline development and deployment).
- **6. Project Management (Week 13):** Methods and tools for managing software projects. Agile methodologies, teamwork, and planning a full-stack project from start to finish.
- **7. Deployment Strategies (Week 14):** How to deploy applications to production. Includes discussion of cloud hosting, containers (Docker), deployment pipelines, and other strategies.

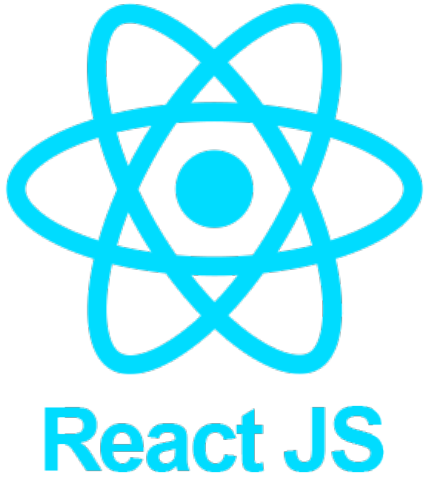
LABORATORY PROJECT

- For your laboratory project, work in teams of 2 students. You will **design and build a website of your choice** using the skills presented throughout the course. Your website can serve any purpose (e.g., a personal portfolio, a blog, an online store, a community site, or a web app), but it should demonstrate the following:
  - **Front-end**: A clear and functional user interface built with modern front-end technologies.
  - **Back-end**: A working server-side component that handles data or user interactions.
  - **Database**: Integration with a database (SQL or NoSQL) to store and retrieve information.
  - **Version Control & DevOps:**
    - The project must be developed in a **GitHub repository** with **at least 10 meaningful commits**.
    - Use **at least one Pull Request** from a feature branch into main.
    - Include a **GitHub Actions CI workflow** that runs tests and/or linting on every push or pull request.
- You will be graded on creativity, functionality, use of technologies covered in the course and DevOps practices.

FRONT-END TECHNOLOGIES

- The **front-end** of a web application deals with everything the user sees and interacts with directly. It's implemented with web technologies: **HTML** for content structure, **CSS** for presentation (layout and styling), and **JavaScript** for interactivity.
- **User Experience:** ensuring an intuitive and responsive user experience. This involves responsive design (so that apps work on mobile and desktop), accessibility considerations, and performance optimizations (so pages load quickly and feel responsive).

FRONT-END TECHNOLOGIES



JAVASCRIPT

- JavaScript is the dominant programming language for the web's front-end. It runs in all modern browsers and enables interactive features on webpages.
- JavaScript is a high-level, dynamic language that supports event-driven, asynchronous programming (e.g., handling user clicks or fetching data in the background).
- It manipulates the **DOM (Document Object Model)** to change page content and responds to user actions.
- While initially a front-end language, JS can also run on servers (thanks to Node.js) and even on desktops (Electron) or mobile (React Native).

REACT

- React is a **free, open-source front-end JavaScript library** for building user interfaces using a **component-based** architecture.
- In React, the UI is divided into small, reusable components.
- React introduced the concept of a virtual DOM, which is a lightweight in-memory representation of the DOM. When state changes occur, React efficiently updates and re-renders only the necessary parts of the actual DOM.
- It promotes a unidirectional data flow. Data flows down from parent components to child components via **props**.
- React offers multiple libraries such as: **React Router** (for client-side routing), **Redux or Context API** (for state management across components), and tools like **Create React App / Vite** to set up projects.

ANGULAR

- Angular is a **TypeScript-based open-source web application framework.** It provides: a powerful templating system, built-in two-way **data binding**, form handling and validation, an HTTP client for API calls, routing, and more.
- Similar to React, Angular apps are composed of components. In Angular, components are organized within modules, and you use **decorators** (like @Component) to define them. The templates (HTML) and component logic (TypeScript) are closely linked, often with an associated CSS for styling the component.
- Angular offers the ability to synchronize the model and view. When the model state changes, the UI updates, and vice versa (for example, user input updating the underlying model automatically).

SERVER-SIDE PROGRAMMING

- Server-side development focuses on business logic, data processing, and database operations. It ensures that when a user interacts with the UI, the appropriate actions happen on the server.
- A server-side program handles tasks such as: user authentication, authorization (checking permissions), calculations and decision-making, and interactions with the database. It also defines the **API endpoints** that the front-end or other clients call to perform operations or fetch data.
- Back-end development can be done with many languages: JavaScript/TypeScript (Node.js), Python, Java, C#, Ruby, PHP, etc.
- On the server side, you have to manage multiple clients at once. This involves understanding how your server handles concurrency (e.g., Node's event loop vs. Python's multi-threading or async capabilities).

SERVER-SIDE PROGRAMMING

NODE.JS

- **Node.js is a cross-platform, open-source JavaScript runtime environment** that allows JS code to run outside of a browser (on the server).
- It uses an event-driven, non-blocking I/O model. This means Node can handle many simultaneous connections efficiently by **not waiting** for operations (such as file or database reads) to complete before moving on to the next task.
- Node comes with **npm (Node Package Manager)**, which hosts a large number of libraries and modules. This makes development faster because for almost any functionality (connecting to a database, integrating an API, parsing files, etc.), there's likely an npm package available.
- Node.js is particularly popular for real-time applications (like chat applications, real-time collaboration tools) and APIs.

EXPRESS.JS

- Express is web framework for Node.js. It provides a lightweight set of tools for building web servers and APIs with Node.
- With Express, we can define **routes** (URL endpoints) and attach handler functions to them.
- The middleware system of Express allows us to plug in functionality that runs in the request/response pipeline (e.g., authentication checks, logging, body parsing for JSON) in a modular way.
- Because Express is minimalist, it's quick to get something running. A basic web server can be created in a few lines of code.

PYTHON & DJANGO

- Python is a general-purpose programming language known for its readability and productivity.
- Django is a Python web framework that encourages rapid development and clean, pragmatic design.
- Django follows the Model-View-Controller pattern (MVT: Model-View-Template in Django terminology).
  - The **Model** represents the data schema.
  - The **View** handles request/response logic. It's like the controller in other frameworks, retrieving models and rendering responses.
  - The **Template** is the HTML presentation layer, which can use Django's templating language to dynamically generate HTML based on data.

DATABASE MANAGEMENT

- Database management is a critical component of the stack, responsible for organizing data and ensuring it can be efficiently retrieved, updated, and maintained.
- We classify databases broadly into **SQL (relational)** and **NoSQL (non-relational)**:
  - **Relational databases (SQL):** Use tables with fixed schemas (columns) and relationships between those tables. They rely on SQL (Structured Query Language) for queries. Examples include MySQL, PostgreSQL, Oracle, SQL Server.
  - **Non-relational databases (NoSQL):** Do not require a fixed table schema. They can store unstructured or semi-structured data, often as key–value pairs, documents, wide-column tables, or graphs. Examples include MongoDB (document store), Cassandra (wide-column store), Redis (key-value), Neo4j (graph DB).
- The back-end code communicates with the database through queries. In Node/Express, libraries are used or ORMs (like Sequelize or Mongoose for MongoDB) to interact with databases. In Django, the built-in ORM is used primarily to interact with an SQL database (Django by default supports PostgreSQL, MySQL, etc.).

DATABASE MANAGEMENT

nosql

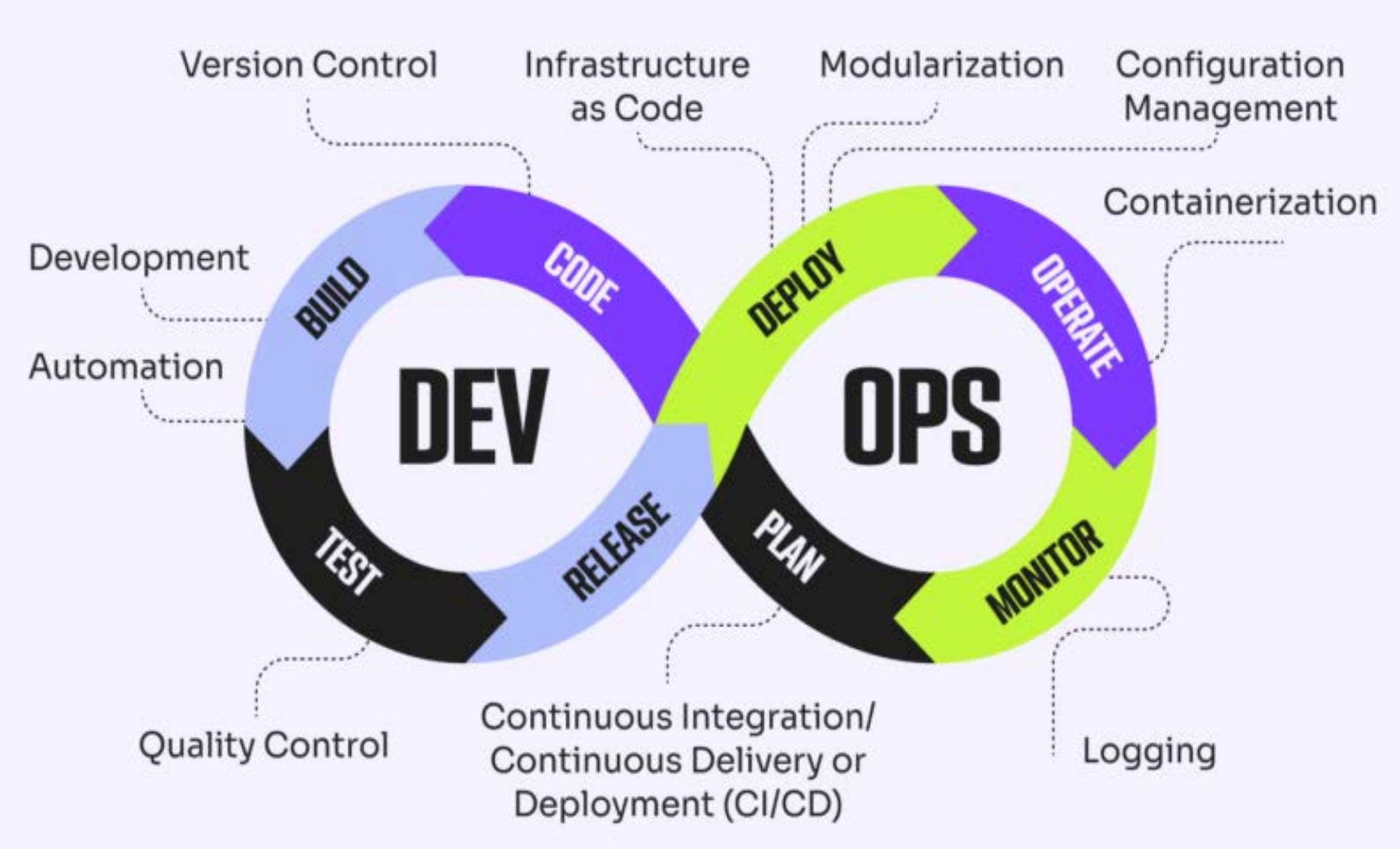
SQL DATABASES (RELATIONAL)

- SQL databases use a structured, table-based data model. Data is stored in tables (which look like spreadsheets with rows and columns). Each table has a defined schema (each column has a data type and purpose), and tables can be linked via **relationships** (one-to-one, one-to-many, many-to-many) using foreign keys.
- We interact with relational databases using SQL.
- With SQL you can **SELECT** data (with powerful filtering, joining multiple tables, aggregating results), **INSERT** new records, **UPDATE** existing ones, and **DELETE** as needed.
- A major feature of relational DBs is support for transactions that are Atomic, Consistent, Isolated, and Durable (ACID). This means a series of database operations can be executed such that they either completely succeed or completely fail, and concurrent operations don't interfere improperly with each other. This is crucial for applications where consistency is key (e.g., financial transactions).

NOSQL DATABASES (NON-RELATIONAL)

- **NoSQL databases are purpose-built for non-relational data models and generally have flexible schemas.**
- There are several types of NoSQL databases:
  - **Document Stores:** (e.g., MongoDB, CouchDB) Store data as documents (often JSON or BSON). This is like storing a bunch of objects or dictionaries. It's great for hierarchical data or aggregates.
  - **Key-Value Stores:** (e.g., Redis, Amazon DynamoDB) Very simple model, like a dictionary or hash map.
  - **Wide-Column Stores:** (e.g., Cassandra, HBase) Use tables, but each row can have a variable number of columns.
- NoSQL databases are often designed to scale out horizontally easily (add more servers to handle more data or traffic).

DEVOPS PRACTICES IN FULL-STACK DEVELOPMENT



DEVOPS PRACTICES IN FULL-STACK DEVELOPMENT

- **DevOps is a set of practices that combines software development and IT operations to deliver software faster and more reliably. Such practices include:**
- **Source Control:** Full-stack developers must be proficient with git for managing code changes, branching, merging, and collaborating via platforms like GitHub or GitLab.
- **Continuous Integration (CI):** This practice involves automatically building and testing the application whenever new code is committed.
- **Continuous Deployment/Delivery (CD):** Taking CI further, CD automates the release of software to production.
- DevOps encourages treating server setup and infrastructure configuration as code (using tools like Docker, Terraform, or Ansible).
- Once deployed, a DevOps mindset means continually monitoring the application (using tools for logging, error tracking, performance monitoring or cloud-specific services).

PROJECT MANAGEMENT FOR DEVELOPMENT

- **Planning and Organization:** Project management, in this context, means breaking down the development process into manageable pieces. We often start with requirements, then plan out features and tasks.
- **Agile Methodologies:** This involves iterative development and frequent reassessment of goals. **Scrum** is a popular Agile framework: teams work in time-boxed sprints (e.g., 2 weeks), plan tasks in a sprint planning meeting, have daily stand-ups to synchronize, and at the end demonstrate the work and gather feedback. **Kanban** is another approach focusing on continuous flow of tasks with a visual board to track progress.
- **Tools for Tracking:** To keep a project on track, teams use tools like **Jira**, **Trello**, or **Asana** to create user stories or tasks, assign them, and track their status.
- **Team Communication:** Effective project management ensures communication between team members. This might include regular meetings (stand-ups, sprint reviews, retrospectives in Scrum) and using communication tools (Slack, Microsoft Teams) to discuss issues quickly.
- **Quality and Adjustments:** Project management also involves keeping an eye on quality (through code reviews, testing, and possibly QA processes) and handling changes.

DEPLOYMENT STRATEGIES

- **From Code to Production:** Deployment is the process of releasing your application to a live environment where end-users can access it.
- Typically, applications move through stages – *development* (on your local machine), maybe a *staging/testing* environment, and then *production*. Each environment should be as similar as possible to avoid "it worked on my machine" issues.
- There are many ways to host and deploy: On-Premises Servers, Cloud Infrastructure (IaaS), Platform as a Service (PaaS), Serverless & Functions.